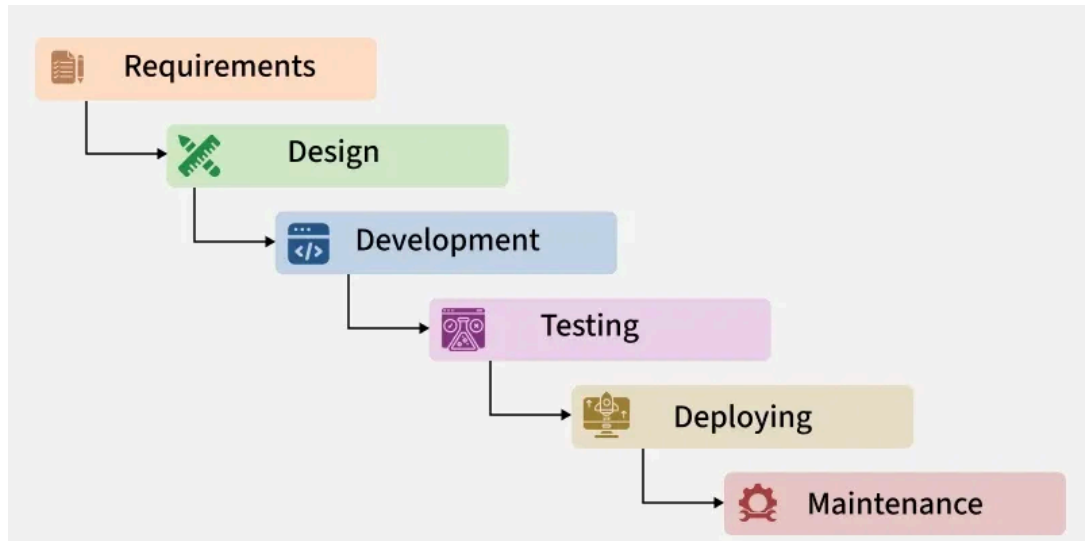


Software Engineering Study Guide

SECTION-A

Introduction to Software Engineering

- Software Problem
 - Refers to the difficulties encountered in developing software due to increasing scale and complexity.
 - Symptoms include schedule delays, cost overruns, poor quality, and unmet user requirements.
 - Driven by continuous changes in technology and user expectations.
- Software Engineering Approach
 - The application of a systematic, disciplined, and quantifiable approach to the development, operation, and maintenance of software.
 - Focuses on using proven engineering principles rather than ad-hoc coding.
 - Emphasizes phased development, documentation, and quality assurance.
- Software Process
 - A framework of tasks, activities, and actions required to build high-quality software.
 - Includes generic phases: Communication, Planning, Modeling, Construction, and Deployment.
- Characteristics of Software Engineering Process
 - Predictability: The process should accurately estimate cost and time.
 - Testability: The output should be easily verifiable against requirements.
 - Maintainability: The resulting software must be easy to update and fix.
 - Defect Removal Efficiency: The process must catch errors early in the lifecycle.
- Software Development Models
 - **Waterfall Model:** A linear, sequential approach where each phase must be completed before the next begins. Good for clear, unchanging requirements.

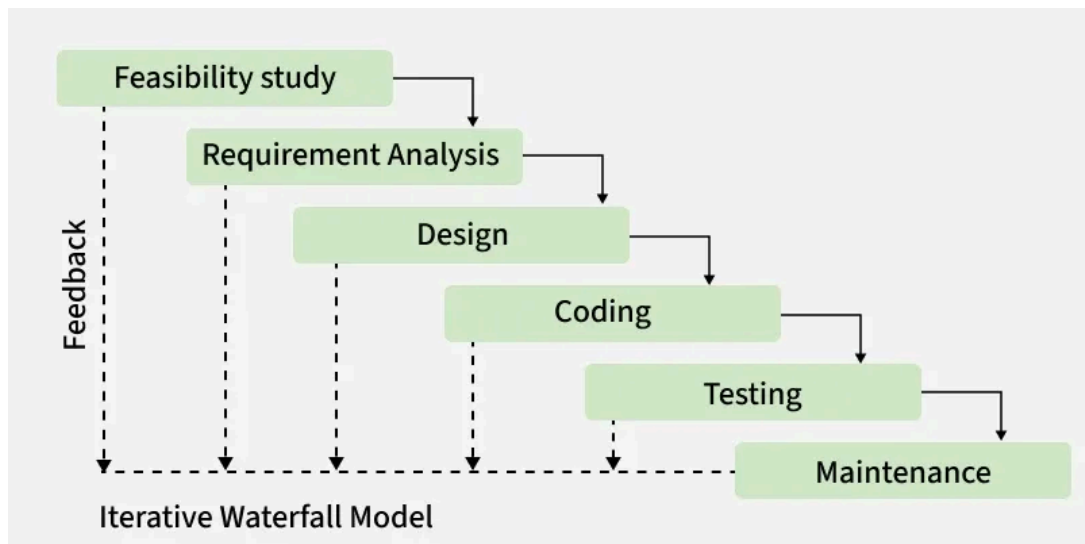


Alpha Testing: Performed by the development team.

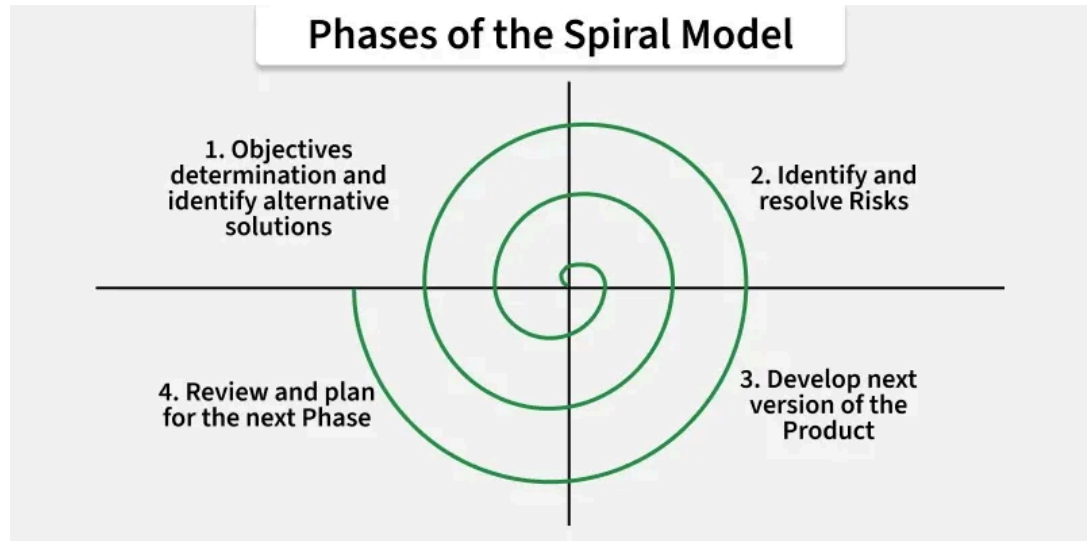
Beta Testing: Performed by selected end users.

Acceptance Testing: Performed by the customer to approve the software.

- Iterative Model: Develops the system through repeated cycles (iterations) and in smaller portions at a time.



- Spiral Model: Combines iterative development with systematic aspects of the waterfall model, heavily focusing on risk analysis.



- Agile Model: Promotes continuous iteration, flexibility, and customer collaboration throughout the development lifecycle.

Software Requirement Analysis and Specification

- Problem Analysis
 - The process of understanding the real-world problem that the software needs to solve.
 - Involves interviewing stakeholders, observing current systems, and identifying constraints.
 - Goal is to organize information to eliminate misunderstandings between clients and developers.
- Software Requirement Specification (SRS)
 - A formal document that acts as a contract between the client and the development team.
 - Describes what the software will do, but not how it will do it.
- Requirement Specifications (Characteristics of a good SRS)
 - Correct: Every requirement is one that the software shall meet.
 - Unambiguous: Every requirement has only one interpretation.
 - Complete: Includes all significant requirements and responses to all input classes.
 - Consistent: Requirements do not conflict with one another.
 - Verifiable: There exists a finite, cost-effective process to check if the software meets the requirement.

Software Project Planning

- **Cost Estimation**
 - The process of predicting the financial resources required to develop the software.
 - Involves estimating hardware, software tools, human resources, and facility costs.
- **Cost Estimation Models**
 - **Expert Judgment:** Relying on the experience of senior engineers.
 - **Algorithmic Models (e.g., COCOMO):** Uses mathematical equations based on historical data and project variables (like Lines of Code) to compute cost and effort.

Comparison of Types of Projects in COCOMO Model

Here is the Comparison in detail where the project types of COCOMO Model

Aspects	Organic	Semidetached	Embedded
Project Size	2 to 50 KLOC	50-300 KLOC	300 and above KLOC
Complexity	Low	Medium	High
Team Experience	Highly experienced	Some experienced as well as inexperienced staff	Mixed experience, includes experts
Environment	Flexible, fewer constraints	Somewhat flexible, moderate constraints	Highly rigorous, strict requirements
Effort Equation	$E = 2.4(400)^{1.05}$	$E = 3.0(400)^{1.12}$	$E = 3.6(400)^{1.20}$
Example	Simple payroll system	New system interfacing with existing systems	Flight control software

- **Project Scheduling**
 - Allocating the estimated effort across the planned project duration.
 - Identifying task dependencies and milestones.
 - Tools used include Gantt charts and PERT/CPM networks.
- **Software Configuration Management (SCM)**
 - The discipline of evaluating, coordinating, and approving changes to

- software artifacts.
 - Version Control: Managing multiple versions of code and documents.
 - Change Control: A formal process for submitting and reviewing change requests.
- Team Structure
 - Hierarchical: Clear chain of command, good for large projects.
 - Democratic/Egoless: Decisions made by consensus, good for complex, research-oriented problems.
 - Chief Programmer: A highly skilled leader directs a team of specialists.
- Risk Management
 - Risk Identification: Recognizing potential threats to project success (e.g., staff turnover, changing tech).
 - Risk Analysis: Assessing the probability and impact of each risk.
 - Risk Mitigation: Developing strategies to reduce the likelihood or impact of the risk.

SECTION-B

Function Oriented Design

- Design Principles
 - Focuses on dividing a system into smaller, manageable functions or modules.
 - Top-down decomposition is the primary strategy.
- Coupling
 - The degree of interdependence between different modules.
 - Low Coupling is desired. It means modules are independent; a change in one rarely affects others.
- Cohesion
 - The degree to which elements inside a single module belong together.
 - High Cohesion is desired. It means a module performs one single, specific task perfectly.
- Structured Design Methodologies
 - Uses Data Flow Diagrams (DFDs) to model the flow of data through the system.
 - Translates the DFD into a Structure Chart, which shows the hierarchical organization of modules.

Object Oriented Design

- Object-Oriented Analysis and Design (OOAD)
 - A software engineering approach that models a system as a group of interacting objects.
 - Combines data and the operations that manipulate that data into a single

entity.

- **Classes and Objects**
 - **Class:** A blueprint or template defining the properties (attributes) and behaviors (methods) of a specific entity.
 - **Object:** An instance of a class. It occupies memory and has a specific state.
- **Inheritance**
 - A mechanism where a new class (child) inherits attributes and methods from an existing class (parent).
 - Promotes code reusability and establishes an "is-a" relationship.
- **Polymorphism**
 - The ability of different objects to respond to the same method call in their own specific way.
 - Allows for flexibility and dynamic behavior at runtime.
- **Design Notation and Specification**
 - **UML (Unified Modeling Language):** The standard visual language for OOAD.
 - Uses Class Diagrams, Use Case Diagrams, and Sequence Diagrams to specify system architecture.

Standards Related to Software Engineering

- **ISO 9001:** An international standard for quality management systems. Focuses on continuous improvement and customer satisfaction.
- **CMMI (Capability Maturity Model Integration):** A framework that evaluates the maturity of a company's software development processes across five levels (from Initial to Optimizing).
- **IEEE Standards:** Provides specific guidelines for various software engineering documents and processes (e.g., IEEE 830 for SRS documents, IEEE 1012 for Verification and Validation).

Coding

- **Top-Down Approach**
 - Starts with the high-level main function and gradually breaks it down into lower-level sub-functions.
 - Uses "stubs" (dummy modules) to test higher-level logic before lower-level modules are written.
- **Bottom-Up Approach**
 - Starts by coding and testing the lowest-level utility functions first.
 - Uses "drivers" to test these modules, integrating them upwards to build the main program.
- **Structured Programming**
 - A programming paradigm aimed at improving clarity and quality by using control structures like sequence, selection (if/else), and iteration (loops).
 - Avoids the use of unstructured jumps like "GOTO" statements.
- **Information Hiding**

- The principle of concealing the internal workings of a module from the rest of the system.
- Only exposing essential interfaces, reducing system complexity and preventing unintended interference.
- **Programming Style**
 - Adhering to standard naming conventions, proper indentation, and consistent formatting to make the code readable and maintainable by others.
- **Documentation**
 - Internal Documentation: Comments within the source code explaining complex logic.
 - External Documentation: User manuals, installation guides, and API references.

Testing

- **Testing Fundamentals**
 - The process of executing a program with the intent of finding errors.
 - Testing proves the presence of defects, not their absence.
- **White Box Testing**
 - Also known as structural or glass-box testing.
 - The tester has full knowledge of the internal source code and logic.
 - Focuses on testing specific paths, loops, and conditional statements.
- **Black Box Testing**
 - The tester has no knowledge of the internal code.
 - Focuses entirely on the inputs and expected outputs based on the software requirements.
- **Functional Testing**
 - A type of black-box testing that verifies if the software features work according to the functional requirements.
- **Boundary Value Testing**
 - A black-box technique that tests the extreme edges of input ranges.
 - Errors are most likely to occur at the boundaries rather than the center of an input domain (e.g., testing 1 and 100 for a valid range of 1-100).
- **Cause-Effect Graphing**
 - A black-box testing technique that illustrates the relationship between a given outcome (effect) and all factors that influence that outcome (causes).
 - Helps in designing test cases that cover complex combinations of conditions.